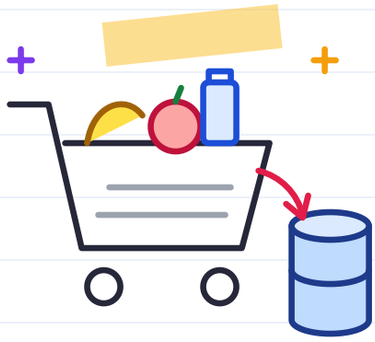
my SQL notebook

SQL from Zero



Hand-drawn notes, supermarket edition

Zero experience needed. Every idea is explained slowly, with a real example and its real answer, so you can learn without having to ask anyone.

What is a database?

A **database** is an organised place where data is stored, so it can be found again quickly.

Picture a supermarket's back office: one neat list of every product, one of every customer, one of every order. Each list is a table. All the lists together = the database.

What is SQL?

SQL = Structured Query Language. A *query* is just a question. SQL is the language you use to ask a database questions, and to add, change or delete data.

Say it **"S-Q-L"** or **"sequel"**. Both are correct.

One language, many database programs

MySQL · PostgreSQL · SQL Server · SQLite · Oracle ...all speak SQL.

They have tiny "accent" differences (like British vs American English). These notes use the common standard, and point out the differences when they matter.

How you "talk" to a database

"Which products are **out of stock**?"

```
SELECT product_name
FROM products
WHERE stock = 0;
```

product_name
Orange Juice

1. question in English
2. the same question in SQL
3. answer = a small table

How to read these notes

- SELECT blue = SQL keyword
- products orange = table name
- 'Dairy' green = a value
- COUNT teal = function
- -- note grey = comment

Under each query, look for **In plain English**. It says what the query means, word by word.

The road map

1. **Foundations** data, tables, grammar · p.2-4
2. **Reading data** SELECT to LIMIT · p.5-11
3. **Summarising** totals, groups · p.12-14
4. **Combining** joins and more · p.15-19
5. **Building** create, change · p.20-22
6. **Review** cheat sheet, practice · p.23-25

☆ ***Tip:*** learning sticks when you type. Paste the dataset (page 2) into a free online editor such as W3Schools' "Try it Yourself", sqliteonline.com or DB Fiddle, and run every example yourself.

02 Meet FreshMart, our data

A small supermarket with 4 tables. Every example in these notes uses them.

products — everything on the shelves (12 rows)

product_id	product_name	category	price	stock	supplier_id
1	Bananas	Fruit	1.20	150	1
2	Whole Milk	Dairy	1.10	80	2
3	Cheddar Cheese	Dairy	3.50	40	2
4	Sourdough Bread	Bakery	2.80	25	3
5	Croissant	Bakery	0.90	60	3
6	Apples	Fruit	2.40	120	1
7	Orange Juice	Drinks	3.20	0	4
8	Sparkling Water	Drinks	0.60	200	4
9	Dark Chocolate	Snacks	1.99	75	NULL
10	Potato Chips	Snacks	1.49	90	5
11	Greek Yogurt	Dairy	0.79	50	2
12	Strawberries	Fruit	3.99	30	1

price is in euros (€)

↙ Orange Juice: stock is 0, sold out!

↙ Dark Chocolate: supplier unknown, so the cell is empty (NULL). See page 10.

5 categories: Fruit, Dairy, Bakery, Drinks, Snacks

customers — people who shop here

customer_id	first_name	city
1	Anna	Aachen
2	Ben	Cologne
3	Chloe	Aachen
4	David	Berlin
5	Emma	Cologne

Emma signed up but never ordered (page 16)

suppliers — who delivers the goods

supplier_id	supplier_name	country
1	Green Farm	Germany
2	Alpine Dairy	Austria
3	Baker's Best	Germany
4	Splash Drinks	Netherlands
5	Crunch Co	Belgium
6	Nordic Fish	Norway

Nordic Fish supplies nothing yet

orders — one row = one product bought by one customer

order_id	customer_id	product_id	quantity	order_date
101	1	1	6	2026-10-01
102	1	2	2	2026-10-01
103	2	4	1	2026-10-02
104	3	8	12	2026-10-03
105	4	3	1	2026-10-03
106	2	1	3	2026-10-05
107	3	10	2	2026-10-06
108	1	12	1	2026-10-07

How to read row 101:

customer **1** (Anna) bought product **1** (Bananas), **6** of them, on 1 Oct 2026.

The orders table only stores **ids**, not names. To see names we "join" tables (page 15).

dates are written YYYY-MM-DD

☆ **Bookmark this page.** Whenever an example gives an answer, flip back here and check it with your own eyes. That is the fastest way to trust SQL.

03 Tables, rows, columns & keys

The words you'll hear all the time, explained once and properly

Anatomy of a table

table name: products

product_id	product_name	category	price
1	Bananas	Fruit	1.20
2	Whole Milk	Dairy	1.10
6	Apples	Fruit	2.40
8	Sparkling Water	Drinks	0.60

(only 4 of the 12 rows shown)

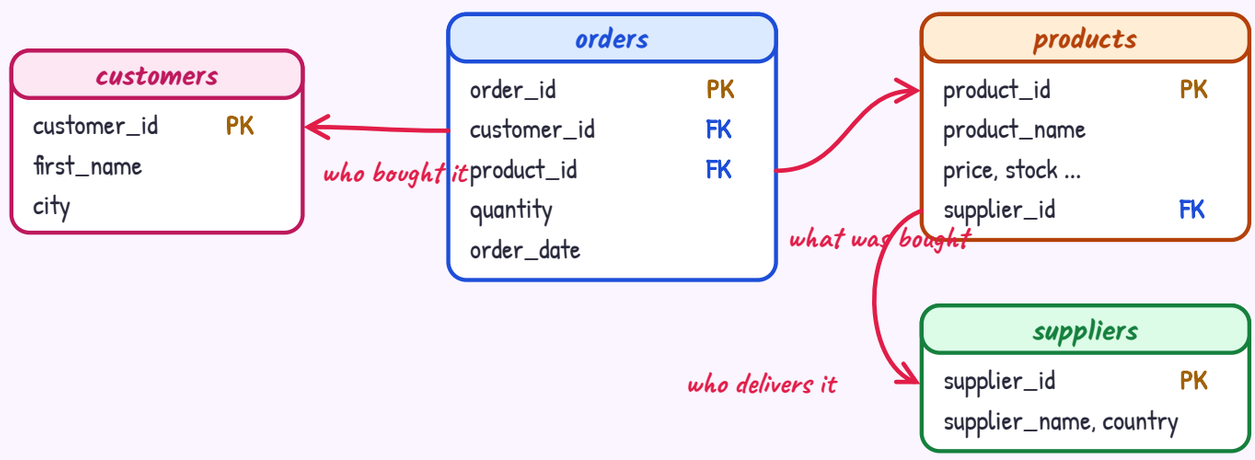
- **Header**: the column names. They describe what's stored below.
- **Column** (also called **field**): one kind of information, going down. The *price* column holds every price.
- **Row** (also called **record**): one complete thing, going across. This row is everything about Apples.
- **Cell** (or **value**): where a row meets a column. Apples' price = 2.40.

Data type: each column holds only one kind of data. *price* holds decimal numbers, *product_name* holds text, *order_date* holds dates. You can't put "banana" in the price column. (More on page 20.)

Keys: how tables point to each other

Primary key (PK): a column that gives every row a unique id. It is never empty and never repeats. Like the **barcode** on each product: two products can share a name, never a barcode.

Foreign key (FK): a column that stores *another* table's primary key, to say "this row belongs to that row". *orders.customer_id = 1* means "this order belongs to customer 1, Anna".



Why 4 small tables and not 1 giant one?

If everything were in one table, Anna's name and city would be typed again on every order she makes. If she moves city, you'd have to fix it in many places, and you'd probably miss one.

With separate tables, Anna is stored once in *customers*. Orders just point to her id. Fix it once, correct everywhere. This tidy design is called **normalisation**.

☆ ***Takeaway:*** rows go across, columns go down. A primary key uniquely names a row; a foreign key points to a row in another table. Keys are what make joins possible later.

Part 1 · Foundations

04 SQL grammar rules

SQL reads almost like an English sentence. Here's how the sentence is built.

Anatomy of one SQL statement

SELECT	product_name, price	FROM	products	WHERE	price > 2	;
keyword "show me"	column list (comma between)	keyword "look in"	table name	keyword "only if"	condition (a yes/no test)	full stop

Each keyword starts a **clause**: the SELECT clause, the FROM clause, the WHERE clause. The whole thing, up to the ;, is a **statement** (or **query** when it asks a question).

In plain English: "Show me the product name and price, from the products table, only for products that cost more than 2 euros."

8 rules worth knowing

1. **Keywords ignore case.** SELECT, select and SeLeCt all work. Convention: write keywords in CAPITALS so they stand out.
2. **End with a semicolon ;** It marks where one statement stops. Some tools forgive you if it's missing; it's needed when you run several statements at once.
3. **Spaces and new lines don't matter.** Put each clause on its own line. It's easier to read and to fix.
4. **Text and dates go in 'single quotes'.** 'Dairy', '2026-10-01'. **Numbers don't:** 3.50, 150.
5. **Decimals use a dot:** 3.50, never ~~3,50~~. (Careful if you're used to German or European number formats!)
6. **Names without spaces.** Use underscores: *product_name*, *order_date*. This style is called snake_case.
7. **Text values may care about case.** 'Dairy' vs 'dairy': MySQL and SQL Server usually treat them as equal, PostgreSQL doesn't. Safest: match the data exactly.
8. **Comments are ignored by SQL.** *-- one line* or */* several lines */*. Use them to leave notes for yourself.

Same query, two styles

works, but hard to read

```
select product_name,price from products
where price>2;
```

same result, much clearer

```
-- products over 2 euros
SELECT product_name, price
FROM   products
WHERE  price > 2;
```

The 3 families of SQL commands

Ask questions	Change the data	Build the structure
SELECT read data, change nothing. 80% of your SQL life. (p.5–19)	INSERT · UPDATE · DELETE add, edit, remove rows (p.21)	CREATE · ALTER · DROP make, reshape, delete tables (p.20–21)

☆ ***Takeaway:*** a statement is built from clauses, each starting with a keyword. CAPS for keywords, one clause per line, quotes around text, dot for decimals, ; at the end.

05 SELECT: choose your columns

The most used command in SQL. It reads data and never changes it.

The recipe

```
SELECT
column1,
column2
FROM
table_name;
```

SELECT = which columns do you want to see?

FROM = which table are they in?

The database reads FROM first (find the table), then SELECT (pick the columns), even though you write SELECT first.

product_id	product_name	category	price	stock
1	Bananas	Fruit	1.20	150
2	Whole Milk	Dairy	1.10	80
3	Cheddar Cheese	Dairy	3.50	40

→

product_name	price
Bananas	1.20
Whole Milk	1.10
Cheddar Cheese	3.50

SELECT lifts out just the purple columns

Ex 1 · everything

```
SELECT * FROM products;
```

In plain English: show every column of every product.

* (asterisk, "star") means all columns. Result: the whole 12-row table from page 2.

Great for a first look. In real reports, name the columns you need: faster and clearer.

Ex 2 · one column

```
SELECT product_name FROM products;
```

In plain English: list just the names of all products.

result (12 rows): Bananas · Whole Milk · Cheddar Cheese · Sourdough Bread · Croissant · Apples · ...6 more

Ex 3 · column order is up to you

```
SELECT price, product_name, stock
FROM products;
```

In plain English: price first, then name, then stock.

price	product_name	stock
1.20	Bananas	150
1.10	Whole Milk	80
... 10 more rows		

Columns appear in the order you list them. The table itself is unchanged.

Ex 4 · a different table

```
SELECT first_name, city
FROM customers;
```

In plain English: who are our customers and where do they live?

first_name	city
Anna	Aachen
Ben	Cologne
Chloe	Aachen
David	Berlin
Emma	Cologne

⚠ Classic beginner errors (and what they mean)

~~SELECT product_name price~~ → missing comma. SQL thinks "price" is a nickname for product_name (see p.6)!

~~SELECT product_name, FROM ...~~ → a comma right before FROM. Delete it.

~~SELECT name FROM products~~ → error "unknown column". The real column is **product_name**.

~~SELECT * FORM products~~ → typo. Errors usually point near the word that broke.

Takeaway: SELECT picks columns, FROM picks the table. Commas between columns, never after the last one. * = all columns.

06 AS, maths & DISTINCT

Rename columns, calculate new ones, and remove duplicates

1. AS: give a column a nickname (an "alias")

```
SELECT product_name AS item,
       price AS price_eur
FROM products;
```

item	price_eur
Bananas	1.20
Whole Milk	1.10
... 10 more	

In plain English: show names and prices, but label the columns "item" and "price_eur".

The alias only changes the **label in the answer**. The real table keeps its column names.

The word AS is optional (*price price_eur* also works) but writing it makes your intent obvious.

Alias with spaces? Wrap it in double quotes: AS "Price in EUR".

2. Do maths inside SELECT (calculated columns)

+	-	*	/	%
add	subtract	multiply	divide	remainder

brackets ( ) work just like in school maths

```
SELECT product_name, price, stock,
       price * stock AS stock_value
FROM products;
```

In plain English: how many euros of each product sit on the shelf?

product_name	price	stock	stock_value
Bananas	1.20	150	180.00
Whole Milk	1.10	80	88.00
Cheddar Cheese	3.50	40	140.00
Orange Juice	3.20	0	0.00
... 8 more			

```
-- 20% off sale
SELECT product_name,
       ROUND(price * 0.8, 2) AS sale_price
FROM products;
```

In plain English: what would each item cost at 20% off, rounded to cents?

product_name	sale_price
Bananas	0.96
Whole Milk	0.88
Cheddar Cheese	2.80
... 9 more	

ROUND(number, 2) = round to 2 decimal places.

Gotcha: in SQL Server and PostgreSQL, 7 / 2 with whole numbers gives 3, not 3.5! Write 7.0 / 2 to get 3.5.

3. DISTINCT: show each value only once

```
SELECT category FROM products;
SELECT DISTINCT category FROM products;
```

In plain English: which categories exist? (no repeats please)



12 rows shrink to 5. With several columns, DISTINCT removes rows where **the whole combination** repeats. Another example: SELECT DISTINCT city FROM customers; → Aachen, Cologne, Berlin.

Takeaway: AS renames, + - * / create new calculated columns, DISTINCT removes repeated rows. None of these change the stored data.

07 WHERE: keep only the rows you want

SELECT picks columns. WHERE picks rows.

How WHERE really works

```
SELECT product_name, price
FROM products
WHERE price > 3;
```

The database walks down the table and asks **every row** the same yes/no question: *"is your price more than 3?"* Only "yes" rows make it into the answer.

product_name	price	price > 3 ?
Bananas	1.20	no
Whole Milk	1.10	no
Cheddar Cheese	3.50	YES
Sourdough Bread	2.80	no
Croissant	0.90	no
Apples	2.40	no
Orange Juice	3.20	YES
Sparkling Water	0.60	no
Dark Chocolate	1.99	no
Potato Chips	1.49	no
Greek Yogurt	0.79	no
Strawberries	3.99	YES

Result: 3 rows. Cheddar Cheese, Orange Juice, Strawberries.

Comparison operators

sign	meaning	example
=	equal to	stock = 0
<>	not equal (also !=)	category <> 'Fruit'
>	greater than	price > 3
<	less than	stock < 50
>=	greater or equal	quantity >= 2
<=	less or equal	price <= 1

Note: equals is ONE = sign in SQL (not == like in Python).

Where WHERE goes

SELECT → FROM → WHERE. Always right after the table name.

You can filter on a column you don't display: *SELECT product_name ... WHERE stock = 0* works fine.

Ex 1 · text: quotes!

```
SELECT product_name FROM products
WHERE category = 'Dairy';
```

In plain English: which products are dairy?

result: Whole Milk · Cheddar Cheese · Greek Yogurt

Ex 2 · running low

```
SELECT product_name, stock FROM
products
WHERE stock < 50;
```

In plain English: what do we need to re-order soon?

result: Cheddar 40 · Sourdough 25 · Orange Juice 0 · Strawberries 30

Ex 3 · not equal

```
SELECT first_name FROM customers
WHERE city <> 'Aachen';
```

In plain English: customers who don't live in Aachen.

result: Ben · David · Emma

Ex 4 · dates

```
SELECT order_id, order_date FROM
orders
WHERE order_date >= '2026-10-05';
```

In plain English: orders from 5 October onwards.

result: 106 · 107 · 108 (later date = "bigger")

Takeaway: WHERE tests each row with a yes/no condition and keeps the "yes" rows. Text and dates need quotes; numbers don't.

08 AND, OR, NOT: combine rules

One condition is rarely enough. These three words glue conditions together.



AND

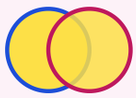
both must be true

```
SELECT product_name, price
FROM products
WHERE category = 'Dairy'
  AND price < 2;
```

In plain English: cheap dairy products (under 2 €).

product	Dairy?	< 2?	AND
Whole Milk	yes	yes	YES
Cheddar Cheese	yes	no	no
Greek Yogurt	yes	yes	YES
Bananas	no	yes	no

result: Whole Milk 1.10 · Greek Yogurt 0.79



OR

at least one is true

```
SELECT product_name, category
FROM products
WHERE category = 'Bakery'
   OR category = 'Snacks';
```

In plain English: everything from the bakery or snack aisle.

rule A	rule B	A AND B	A OR B
yes	yes	yes	yes
yes	no	no	yes
no	yes	no	yes
no	no	no	no

result: Sourdough Bread · Croissant · Dark Chocolate · Potato Chips

NOT flips the answer ⊘

```
SELECT product_name FROM products
WHERE NOT category = 'Drinks';
```

In plain English: everything except drinks.

Result: 10 rows (all but Orange Juice and Sparkling Water). Same as *category <> 'Drinks'*. NOT shines later with IN, LIKE and BETWEEN: *NOT IN*, *NOT LIKE*.

Three rules at once

```
SELECT product_name FROM products
WHERE category = 'Fruit'
  AND price > 2
  AND stock >= 100;
```

In plain English: pricier fruit that we have plenty of.

result: Apples (Strawberries fails: stock 30)

⚠ The bracket trap: AND is done before OR

Just like × is done before + in maths (2 + 3 × 4 = 14), SQL does **AND** before **OR**. Goal: *"fruit or dairy, but only under 1 €"*.

without brackets (wrong!)

```
WHERE category = 'Fruit'
   OR category = 'Dairy' AND price < 1
```

SQL reads: Fruit **OR** (Dairy **AND** < 1) → Bananas · Apples · Strawberries · Greek Yogurt. All fruit sneaks in at any price.

with brackets (right)

```
WHERE (category = 'Fruit'
   OR category = 'Dairy')
  AND price < 1
```

Brackets are worked out first → Greek Yogurt (the only fruit or dairy item under 1 €).

Rule of thumb: mixing AND with OR? Always add brackets.



Takeaway: AND narrows (both true), OR widens (either true), NOT flips. Brackets decide what goes together.

Part 2 · Reading data

09 IN, BETWEEN, LIKE: smarter filters

Shortcuts for lists, ranges and text patterns

IN: "is it one of these?"

```
SELECT product_name FROM products
WHERE category IN ('Fruit', 'Bakery');
```

In plain English: products in the fruit or bakery section.

Bananas | Sourdough Bread | Croissant | Apples | Strawberries

IN is a short way of writing many ORs. These are identical:

```
category = 'Fruit' OR category = 'Bakery'
```

Works with numbers too: *product_id* **IN** *(1, 6, 12)* → Bananas, Apples, Strawberries.

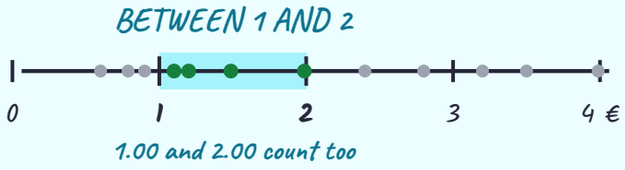
NOT IN excludes the list: *city* **NOT IN** *('Aachen', 'Berlin')* → Ben, Emma.

BETWEEN: "is it inside this range?"

```
SELECT product_name, price FROM
products
WHERE price BETWEEN 1 AND 2;
```

In plain English: products from 1 € up to 2 €, both ends included.

Bananas 1.20 | Whole Milk 1.10 | Dark Chocolate 1.99 | Potato Chips 1.49



Dates work too: *order_date* **BETWEEN** *'2026-10-01'* **AND** *'2026-10-03'* → orders 101 to 105. Small number first! **NOT BETWEEN** gives everything outside.

LIKE: search inside text with wildcards

Use LIKE (not =) when you only know **part** of the text. Two special "joker" characters:

% **any number** of characters (zero, one, or many)

_ **exactly one** character (a single unknown letter)

```
WHERE product_name LIKE 'S%'
```

starts with S → Sourdough Bread | Sparkling Water | Strawberries

```
WHERE product_name LIKE '%Juice'
```

ends with "Juice" → Orange Juice

```
WHERE product_name LIKE '%ee%'
```

contains "ee" anywhere → Cheddar Cheese | Greek Yogurt

```
WHERE product_name LIKE '_pples'
```

one letter, then "pples" → Apples

NOT LIKE keeps the rows that don't match. *Careful: in PostgreSQL LIKE is case-sensitive ('s%' misses "Sourdough"); use ILIKE there to ignore case.*

> ***Takeaway:*** **IN** = matches a list, **BETWEEN** = inside a range (ends included), **LIKE** = text pattern with **%** (many) and **_** (one). Put **NOT** in front to reverse any of them.

10 NULL: the "we don't know" value

The sneakiest idea in SQL. Read this page slowly.

NULL is not zero, and not blank

0
a number
Orange Juice stock = 0: "we counted, there are none left". A real, known value.

''
empty text
A note field left blank on purpose: "nothing to say". Also a real, known value.

NULL
unknown / missing
Dark Chocolate's supplier_id: "nobody wrote it down". There is no value at all.

⚠ The #1 trap: you can't use = with NULL

looks right, returns NOTHING

```
SELECT product_name FROM products
WHERE supplier_id = NULL;
```

Asking "is unknown equal to unknown?" gives the answer **unknown**, not yes. Only "yes" rows pass WHERE, so 0 rows come back. No error, just silence!

the correct way: IS NULL

```
SELECT product_name FROM products
WHERE supplier_id IS NULL;
```

In plain English: products whose supplier is missing.

result: Dark Chocolate

IS NOT NULL → the other 11 products.

Trap #2: NULL rows vanish from <>

```
SELECT product_name FROM products
WHERE supplier_id <> 2;
```

In plain English: products not from Alpine Dairy.

You'd expect 12 − 3 = 9 rows. You get **8**. Dark Chocolate is missing, because "is unknown different from 2?" is unknown. To include it:

```
WHERE supplier_id <> 2 OR supplier_id IS NULL
```

Trap #3: NULL spreads in maths

Any calculation with NULL gives NULL:

```
5 + NULL = NULL
5 * NULL = NULL
```

Makes sense: 5 plus "something unknown" is still unknown.

But **aggregate functions skip NULLs**: COUNT(*) = 12 rows, COUNT(supplier_id) = **11** (page 12).

COALESCE: replace NULL with a fallback value

```
SELECT product_name,
       COALESCE(supplier_id, 0) AS supplier
FROM products;
```

In plain English: show the supplier id, or 0 if it's missing.

COALESCE(a, b, c...) returns the **first value that isn't NULL**. Older cousins: IFNULL (MySQL), ISNULL (SQL Server).

product_name	supplier
Potato Chips	5
Dark Chocolate	0
Greek Yogurt	2
... 9 more	

☆ ***Takeaway:*** NULL means unknown. Test it with IS NULL / IS NOT NULL, never with =. It hides from <>, poisons maths, and COALESCE swaps it for a default.

Part 2 · Reading data

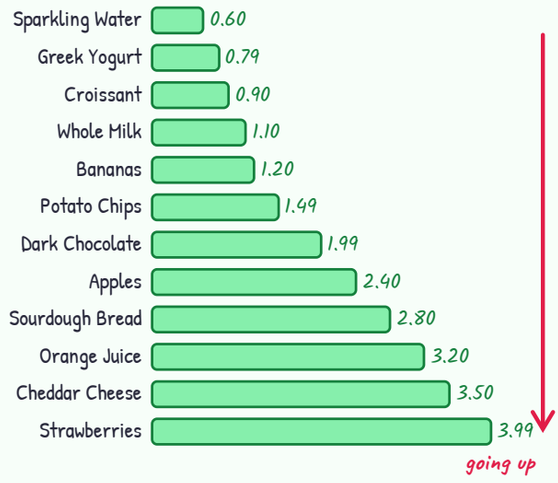
11 ORDER BY & LIMIT

Sort the answer, then keep only the first few rows

ASC = small to big (the default)

```
SELECT product_name, price
FROM products
ORDER BY price ASC;
```

In plain English: all products, cheapest first.



DESC = big to small: *ORDER BY price DESC* flips this list (Strawberries first).

Text sorts A to Z

```
SELECT product_name FROM products
ORDER BY product_name;
```

Apples · Bananas · Cheddar Cheese · Croissant · Dark Chocolate · Greek Yogurt · Orange Juice · Potato Chips · Sourdough Bread · Sparkling Water · Strawberries · Whole Milk

No ASC written? It's ASC anyway. Dates sort oldest → newest.

Sort by two columns

```
SELECT category, product_name, price
FROM products
ORDER BY category ASC, price DESC;
```

In plain English: group by aisle A to Z; inside each aisle, priciest first.

category	product_name	price
Bakery	Sourdough Bread	2.80
Bakery	Croissant	0.90
Dairy	Cheddar Cheese	3.50
Dairy	Whole Milk	1.10
Dairy	Greek Yogurt	0.79
Drinks	Orange Juice	3.20
Drinks	Sparkling Water	0.60
… then Fruit, then Snacks		

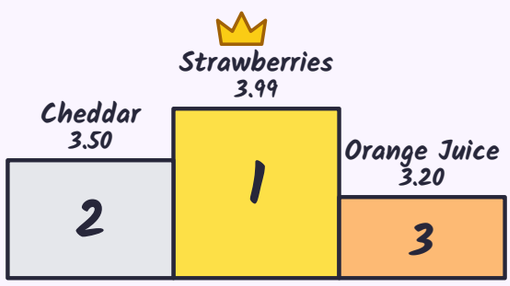
The 2nd column only breaks ties in the 1st.

LIMIT: keep only the first N rows

```
SELECT product_name, price
FROM products
ORDER BY price DESC
LIMIT 3;
```

In plain English: our 3 most expensive products.

Sort **first**, then cut. LIMIT without ORDER BY gives "any 3 rows": databases don't promise any order unless you ask.

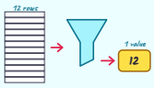


Same idea, different accents: LIMIT 3 · MySQL, PostgreSQL, SQLite · SELECT TOP 3 … · SQL Server · FETCH FIRST 3 ROWS ONLY · Oracle

Takeaway: ORDER BY sorts (ASC up is default, DESC down), can use several columns, and always sits near the end. LIMIT comes last and keeps the top N.

12 COUNT, SUM, AVG, MIN, MAX

"Aggregate functions" squash many rows into a single number



A normal SELECT gives back one row per product. An aggregate function looks at **all the rows together** and returns **one summary value**: how many, the total, the average, the smallest, the biggest. Pattern: `SELECT SUM(stock) FROM products;`

COUNT	SUM	AVG	MIN	MAX
how many rows?	add them up	the average	the smallest	the biggest
`COUNT(*)`	`SUM(stock)`	`AVG(price)`	`MIN(price)`	`MAX(price)`
12	**920**	**2.00**	**0.60**	**3.99**
products	items on shelves	€ (23.96 ÷ 12)	€ (Sparkling Water)	€ (Strawberries)

The exact average is 1.9967; it's shown rounded, as `ROUND(AVG(price), 2)` would give.

Three flavours of COUNT

you write	counts	answ
`COUNT(*)`	all rows	
`COUNT(supplier_id)`	rows where it's not NULL	
`COUNT(DISTINCT category)`	different values	

Dark Chocolate's NULL supplier is why the middle one is 11.

Combine with WHERE

```
SELECT COUNT(*) AS dairy_items
FROM products
WHERE category = 'Dairy';
```

In plain English: how many dairy products do we sell? → **3**

WHERE filters first, then the function counts what's left.

Several at once, with names

```
SELECT MIN(price) AS cheapest,
       MAX(price) AS priciest,
       SUM(price * stock) AS stock_value
FROM products;
```

cheapest	priciest	stock_value
0.60	3.99	1382.55

1382.55 € = the value of everything on our shelves.

MIN and MAX aren't just for numbers

```
SELECT MIN(order_date) AS first_order,
       MAX(order_date) AS last_order
FROM orders;
```

first_order	last_order
2026-10-01	2026-10-07

On text, MIN = first alphabetically (Apples), MAX = last (Whole Milk).

Trap: mixing a summary with a normal column

```
SELECT
product_name,
MAX(price)
FROM products;
-- error!
```

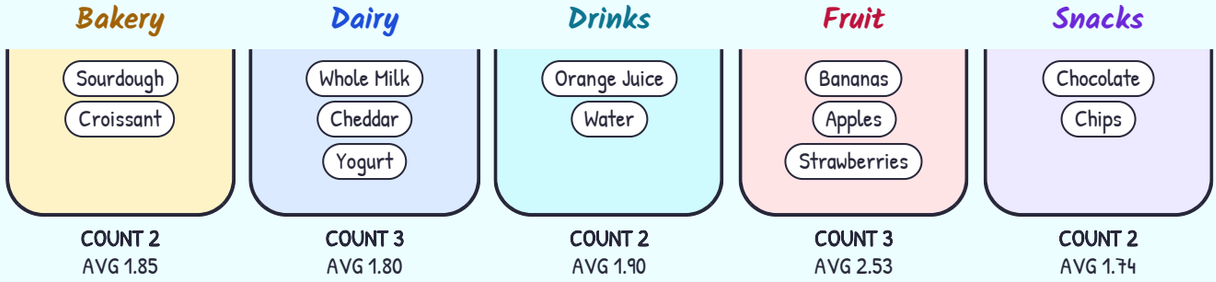
MAX gives **one** row, product_name has **12**. SQL can't line them up. Want the priciest product's name? Use `ORDER BY price DESC LIMIT 1` (p.11), a subquery (p.19), or group the rows (next page).

☆ **Takeaway:** COUNT, SUM, AVG, MIN, MAX turn many rows into one value. They ignore NULLs (except COUNT(*)). Don't mix them with plain columns unless you GROUP BY.

13 GROUP BY: one summary per group

Not "how many products in total?" but "how many products per category?"

Think: sort items into buckets, then measure each bucket



GROUP BY category makes one bucket per category. Then COUNT, AVG, SUM etc. are calculated **inside each bucket separately**. 12 rows in → 5 rows out.

Ex 1 · count per category

```
SELECT category,
       COUNT(*) AS num_products
FROM products
GROUP BY category;
```

In plain English: how many products does each category have?

category	Bakery	Dairy	Drinks	Fruit	Snacks
num_products	2	3	2	3	2

(result drawn sideways to save space: really 5 rows, 2 columns)

Ex 2 · average price, sorted

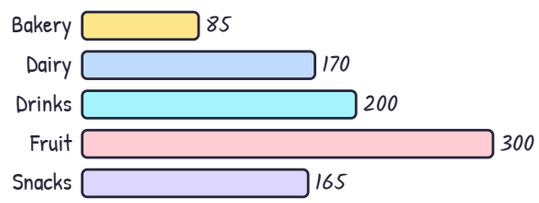
```
SELECT category,
       ROUND(AVG(price), 2) AS
avg_price
FROM products
GROUP BY category
ORDER BY avg_price DESC;
```

In plain English: which aisle is most expensive on average?

category	Fruit	Drinks	Bakery	Dairy	Snacks
avg_price	2.53	1.90	1.85	1.80	1.74

Ex 3 · stock per category

```
SELECT category, SUM(stock) AS items
FROM products
GROUP BY category;
```



Ex 4 · per customer

```
SELECT customer_id,
       COUNT(*) AS orders,
       SUM(quantity) AS items
FROM orders
GROUP BY customer_id;
```

customer_id	1	2	3	4
orders	3	2	2	1
items	9	4	14	1

Only ids for now; names come with JOIN (p.17).

The golden rule of GROUP BY

Every column in SELECT must either be **listed in GROUP BY** or be **inside an aggregate** (COUNT, SUM...).

```
SELECT category, product_name, COUNT(*)
FROM products GROUP BY category;
-- error: which of the 3 dairy names?
```

The Dairy bucket has 3 product names but becomes 1 row. SQL can't choose one for you. Either add it to GROUP BY (more, smaller groups) or remove it.

☆ ***Takeaway:*** GROUP BY makes buckets; aggregates measure each bucket. You get one row per group. Hear "per", "each" or "by" in a question? Think GROUP BY.

14 HAVING & the order SQL thinks in

WHERE filters rows. HAVING filters groups.

Where each filter happens



WHERE can't use COUNT, SUM, AVG... because the buckets don't exist yet when it runs. A condition on an aggregate always goes in **HAVING**.

Ex 1 · busy categories

```
SELECT category, COUNT(*) AS n
FROM products
GROUP BY category
HAVING COUNT(*) > 2;
```

In plain English: categories with more than 2 products.

category	n
Dairy	3
Fruit	3

Bakery, Drinks, Snacks (2 each) were dropped.

Ex 2 · repeat customers

```
SELECT customer_id, COUNT(*) AS
orders
FROM orders
GROUP BY customer_id
HAVING COUNT(*) >= 2;
```

In plain English: who has ordered at least twice?

customer_id	1	2	3
orders	3	2	2

Customer 4 (1 order) was dropped.

Ex 3 · WHERE and HAVING in one query

```
SELECT category, ROUND(AVG(price), 2) AS avg_price
FROM products
WHERE price < 3          -- 1. drop pricey rows
GROUP BY category        -- 2. bucket the rest
HAVING AVG(price) > 1.5; -- 3. drop cheap buckets
```

In plain English: ignoring items of 3 € or more, which categories still average over 1.50 €?

after WHERE, buckets:
Fruit (1.20, 2.40) → 1.80
Dairy (1.10, 0.79) → 0.95
Bakery (2.80, 0.90) → 1.85
Drinks (0.60) → 0.60
Snacks (1.99, 1.49) → 1.74

category	avg_price
Fruit	1.80
Bakery	1.85
Snacks	1.74

The big secret: you write in one order, SQL runs in another

how you WRITE it	how SQL RUNS it
SELECT	5 SELECT
FROM	1 FROM
WHERE	2 WHERE
GROUP BY	3 GROUP BY
HAVING	4 HAVING
ORDER BY	6 ORDER BY
LIMIT	7 LIMIT

Why this matters

SELECT runs 5th. So a nickname made with AS **doesn't exist yet** in WHERE:

```
... price * stock AS value
WHERE value > 100 -- error
```

But ORDER BY runs after SELECT, so *ORDER BY value* works fine.

☆ ***Takeaway:*** WHERE = row filter (before grouping), HAVING = group filter (after, can use aggregates). SQL runs FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.

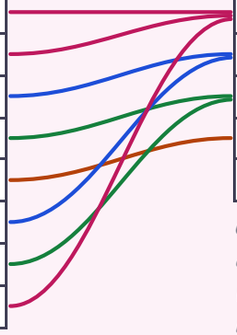
15 JOIN: stitch two tables together

The orders table says "customer 1". We want it to say "Anna".

Matching rows through a shared column

orders

order_id	customer_id
101	1
102	1
103	2
104	3
105	4
106	2
107	3
108	1



customers

customer_id	first_name
1	Anna
2	Ben
3	Chloe
4	David
5	Emma

Emma: no lines, no orders. Watch what happens to her.

What JOIN does, row by row:

1. Take an order, e.g. 103 with customer_id 2.
2. Look in customers for the row whose customer_id is also 2 → Ben.
3. Glue the two rows side by side into one wide row.
4. Repeat for every order. Anna appears 3 times because she has 3 orders.
5. Rows with no partner (Emma) are **left out** of an INNER JOIN.

The query

```
SELECT orders.order_id,
       customers.first_name,
       orders.quantity
FROM orders
INNER JOIN customers
  ON orders.customer_id = customers.customer_id;
```

In plain English: list every order with the name of the person who placed it.

INNER JOIN = which second table to bring in. Writing just JOIN means the same.

ON = the matching rule: which column in one table equals which in the other (usually foreign key = primary key).

table.column = "this column from that table". Needed because both tables have a customer_id.

order_id	first_name	quantity
101	Anna	6
102	Anna	2
103	Ben	1
104	Chloe	12
105	David	1
106	Ben	3
107	Chloe	2
108	Anna	1

Shortcut: table aliases

```
SELECT o.order_id, c.first_name
FROM orders o
JOIN customers c
  ON o.customer_id = c.customer_id;
```

"orders o" gives the table a 1-letter nickname. Same result, far less typing. Professionals almost always do this.

Error: "ambiguous column"

```
SELECT customer_id, first_name
FROM orders o JOIN customers c
  ON o.customer_id = c.customer_id;
```

customer_id lives in **both** tables, so SQL asks "which one?". Fix: write **c.customer_id** (or o.customer_id).

☆ ***Takeaway:*** JOIN ... ON glues rows that share a value (usually FK = PK). INNER JOIN keeps only rows with a match on both sides, so Emma disappears.

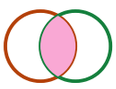
Part 4 · Combining tables

16 LEFT, RIGHT & FULL JOIN

What to do with rows that have no partner

We join **products** (left table, written first) with **suppliers** (right table) **ON** products.supplier_id = suppliers.supplier_id. Two loners:

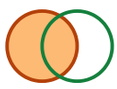
Dark Chocolate: supplier_id is NULL, matches no supplier. **Nordic Fish**: a supplier with no products.



INNER JOIN

11 rows

Only matched pairs. Both loners are dropped.

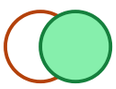


LEFT JOIN

12 rows

All products, matched or not. No supplier? Its columns are filled with NULL.

product_name	supplier_name
Potato Chips	Crunch Co
Dark Chocolate	*NULL*

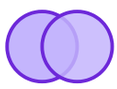


RIGHT JOIN

12 rows

All suppliers, matched or not. The mirror image of LEFT.

product_name	supplier_name
Potato Chips	Crunch Co
NULL	Nordic Fish



FULL OUTER JOIN

13 rows

Everything from both sides: 11 matches + Dark Chocolate + Nordic Fish. (MySQL doesn't support it.)

The LEFT JOIN query, in full

```
SELECT p.product_name, s.supplier_name
FROM products p
LEFT JOIN suppliers s
  ON p.supplier_id = s.supplier_id;
```

Memory trick: the LEFT table is the boss. All its rows stay, no matter what.

In plain English: every product with its supplier's name, even if we don't know the supplier.

"Left" simply means the table written first (after FROM). Swap the two tables and LEFT JOIN acts like RIGHT JOIN. That's why most people only ever use LEFT.

Super useful pattern: find what's MISSING

```
SELECT c.first_name
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

1. LEFT JOIN keeps all 5 customers.
2. Emma has no orders, so her order columns are NULL.
3. WHERE keeps only those NULL rows.

Result: Emma

In plain English: which customers have never placed an order?

Also good to know: a **SELF JOIN** joins a table to itself (e.g. employees and their managers in one table). A **CROSS JOIN** pairs every row with every row (12 products × 6 suppliers = 72 rows), rarely what you want.

Takeaway: INNER = matches only. LEFT = all of the first table + matches. RIGHT = all of the second. FULL = everything. Missing partners show up as NULL.

17 Joining 3 tables for real answers

This is where SQL starts to feel like a superpower



orders holds both ids, so it's the bridge. Just add one **JOIN … ON** per extra table.

Ex 1 · the full receipt

```
SELECT o.order_id, c.first_name, p.product_name,
       o.quantity, p.price,
       o.quantity * p.price AS line_total
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN products  p ON o.product_id  = p.product_id
ORDER BY o.order_id;
```

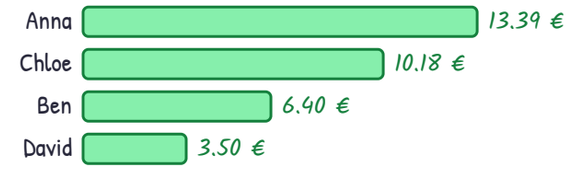
o, c, p = table nicknames (p.15). Each JOIN brings one more table in.

In plain English: for every order, who bought what, how many, at what price, and the total for that line.

order_id	first_name	product_name	quantity	price	line_total
101	Anna	Bananas	6	1.20	7.20
102	Anna	Whole Milk	2	1.10	2.20
103	Ben	Sourdough Bread	1	2.80	2.80
104	Chloe	Sparkling Water	12	0.60	7.20
105	David	Cheddar Cheese	1	3.50	3.50
106	Ben	Bananas	3	1.20	3.60
107	Chloe	Potato Chips	2	1.49	2.98
108	Anna	Strawberries	1	3.99	3.99

Ex 2 · who spends the most?

```
SELECT c.first_name,
       SUM(o.quantity * p.price) AS spent
FROM orders o
JOIN customers c ON o.customer_id =
c.customer_id
JOIN products p ON o.product_id = p.product_id
GROUP BY c.first_name
ORDER BY spent DESC;
```



Check: Anna = 7.20 + 2.20 + 3.99 = 13.39 ✓

Ex 3 · best sellers

```
SELECT p.product_name,
       SUM(o.quantity) AS units
FROM orders o
JOIN products p
  ON o.product_id = p.product_id
GROUP BY p.product_name
ORDER BY units DESC
LIMIT 2;
```

product_name	units
Sparkling Water	12
Bananas	9

Pro habit: add one JOIN at a time and run it each time.

Takeaway: chain one **JOIN … ON** per table, then use everything you know (WHERE, GROUP BY, ORDER BY, LIMIT) on the combined result.

18 CASE & UNION: if-then and stacking

Label rows with your own categories, and pile two results on top of each other

CASE: SQL's "if … then … else"

```
SELECT product_name, price,
  CASE
    WHEN price < 1 THEN 'Budget'
    WHEN price < 3 THEN 'Standard'
    ELSE 'Premium'
  END AS price_band
FROM products;
```

product_name	price	price_band
Bananas	1.20	Standard
Whole Milk	1.10	Standard
Cheddar Cheese	3.50	Premium
Sourdough Bread	2.80	Standard
Croissant	0.90	Budget
Apples	2.40	Standard
Orange Juice	3.20	Premium
Sparkling Water	0.60	Budget
Dark Chocolate	1.99	Standard
Potato Chips	1.49	Standard
Greek Yogurt	0.79	Budget
Strawberries	3.99	Premium

In plain English: tag each product as Budget, Standard or Premium by its price.

price < 1 ? yes → Budget no ↓ price < 3 ? yes → Standard no → Premium

WHENs are checked **top to bottom**; the **first true one wins** and the rest are skipped. Bananas (1.20): not < 1, but < 3 → Standard. No ELSE and nothing matches? You get NULL.

CASE + GROUP BY = instant report

```
SELECT CASE WHEN stock = 0 THEN 'Sold out'
            WHEN stock < 50 THEN 'Low'
            ELSE 'OK' END AS stock_status,
       COUNT(*) AS products
FROM products GROUP BY stock_status;
```

stock_status	products
Sold out	1
Low	3
OK	8

SQL Server can't GROUP BY an alias: repeat the CASE there.

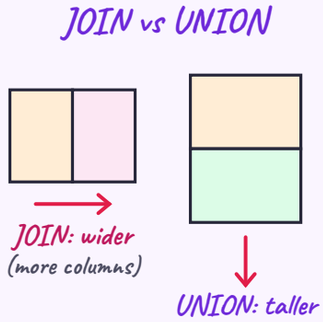
UNION: stack two results on top of each other

```
SELECT city AS place FROM customers
UNION
SELECT country FROM suppliers;
```

In plain English: every place we deal with, customers' cities plus suppliers' countries, no repeats.

Aachen · Cologne · Berlin · Germany · Austria · Netherlands · Belgium · Norway = 8 rows

UNION removes duplicates; UNION ALL keeps them (5 + 6 = 11 rows). **Rule:** both SELECTs need the same number of columns, in the same order, with similar types.



Takeaway: CASE WHEN … THEN … ELSE … END creates labels (first match wins). UNION stacks rows and removes repeats; UNION ALL keeps them.

19 Subqueries: a query inside a query

When the answer to one question is needed to ask the next one



A **subquery** is a complete SELECT written inside brackets ( ), placed inside another query. The inner one runs first; its answer is plugged into the outer one, like solving the brackets first in maths. Inner queries are highlighted like this below.

Ex 1 · above average

```
SELECT product_name, price
FROM products
WHERE price > (SELECT AVG(price)
               FROM products);
```

In plain English: products pricier than the average product.

Step 1: inner → 1.9967. Step 2: WHERE price > 1.9967.

Cheddar 3.50 · Sourdough 2.80 · Apples 2.40 · Orange Juice 3.20 · Strawberries 3.99

Dark Chocolate (1.99) misses by a hair!

Ex 2 · the priciest item's name

```
SELECT product_name
FROM products
WHERE price = (SELECT MAX(price)
               FROM products);
```

In plain English: which product has the highest price?

result: Strawberries

This solves the trap from page 12, and unlike LIMIT 1 it returns **all** products if two share the top price.

Ex 3 · IN with a list

```
SELECT first_name FROM customers
WHERE customer_id IN
  (SELECT customer_id FROM orders
   WHERE product_id = 1);
```

In plain English: who has bought bananas?

Inner → the list (1, 2). Outer → Anna, Ben

Ex 4 · NOT IN: never sold

```
SELECT product_name FROM products
WHERE product_id NOT IN
  (SELECT product_id FROM orders);
```

In plain English: products nobody has ordered yet.

Croissant · Apples · Orange Juice · Dark Chocolate · Greek Yogurt

Careful: if the inner list contains a NULL, NOT IN returns nothing (p.10)!

EXISTS: "is there at least one?"

```
SELECT c.first_name FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o
              WHERE o.customer_id =
c.customer_id);
```

For each customer, check if any order row exists → Anna, Ben, Chloe, David. (NOT EXISTS → Emma.)

Tidier: WITH (a "CTE")

```
WITH per_customer AS (
  SELECT customer_id, SUM(quantity) AS
items
  FROM orders GROUP BY customer_id)
SELECT AVG(items) FROM per_customer;
```

Name a subquery first, then use it like a table. Items 9, 4, 14, 1 → average 7.

★ ***Takeaway:*** a subquery is a SELECT in brackets that runs first. One value → use with = > <. A list → use with IN / NOT IN. Getting long? Name it with WITH.

Part 5 · Building & changing

20 CREATE TABLE: build your own

How the FreshMart tables were made in the first place

Step 0: a database to hold the tables

```
CREATE DATABASE freshmart;    -- make the empty "building"
USE freshmart;                -- MySQL: "work inside this one now"
```

Step 1: describe each column

```
CREATE TABLE products (
  product_id   INT           PRIMARY KEY,
  product_name VARCHAR(100)  NOT NULL,
  category     VARCHAR(50)   NOT NULL,
  price        DECIMAL(6,2)  CHECK (price >= 0),
  stock        INT           DEFAULT 0,
  supplier_id  INT,
  FOREIGN KEY (supplier_id)
    REFERENCES suppliers(supplier_id)
);
```

each line =

- column name
- data type (what fits)
- constraint (a rule)

Commas between lines, none after the last. Brackets wrap the whole list.

Create suppliers first: a foreign key can only point to a table that already exists.

Common data types

type	holds	FreshMart example
INT	whole numbers	stock = 150, product_id = 7
DECIMAL(6,2)	exact decimals: 6 digits total, 2 after the dot (max 9999.99). Use for money.	price = 3.50
VARCHAR(n)	text up to n characters	product_name = 'Apples'
TEXT	long text with no small limit	a product description
DATE	a calendar day, 'YYYY-MM-DD'	order_date = '2026-10-01'
DATETIME	day + time	'2026-10-01 14:30:00'
BOOLEAN	TRUE or FALSE	is_organic = TRUE

Constraints: rules the database enforces for you

PRIMARY KEY
unique + never empty. Blocks a 2nd product with id 7.

NOT NULL
must be filled in. Blocks a product with no name.

UNIQUE
no repeats allowed (e.g. a customer's email), but not the main id.

DEFAULT 0
value used when you don't give one. New products start at stock 0.

CHECK (price >= 0)
custom test. Blocks a price of −2.

FOREIGN KEY
must exist in the other table. Blocks supplier_id = 99.

Auto-numbering ids: MySQL *INT AUTO_INCREMENT*, PostgreSQL *SERIAL*, SQL Server *INT IDENTITY*. The database picks the next id for you.

Takeaway: CREATE TABLE name (column TYPE CONSTRAINT, ...). Types say what fits; constraints stop bad data getting in. Use DECIMAL for money.

Part 5 · Building & changing

21 INSERT, UPDATE, DELETE & friends

Until now we only read. These commands really change the data.

INSERT INTO: add new rows

```
INSERT INTO products (product_id, product_name, category, price, stock, supplier_id)
VALUES (13, 'Salmon Fillet', 'Fish', 6.49, 20, 6);
```

In plain English: add salmon to the shelves. Nordic Fish (supplier 6) finally supplies something!

product_id	product_name	category	price	stock	supplier_id
… rows 1 to 12 …					
13	Salmon Fillet	Fish	6.49	20	6

Values go in the **same order** as the column list. Several rows at once: *VALUES (…), (…), (…);*

Skip a column (e.g. stock) and it gets its DEFAULT or NULL. You can even copy rows in with *INSERT INTO … SELECT …*

UPDATE … SET: change existing values

```
UPDATE products
SET stock = 100
WHERE product_id = 7;
```

In plain English: the juice delivery arrived.

Orange Juice stock: ~~0~~ → 100

```
UPDATE products
SET price = price * 1.10
WHERE category = 'Dairy';
```

In plain English: dairy prices go up 10%.

Milk ~~1.10~~ → 1.21 · Cheddar ~~3.50~~ → 3.85 · Yogurt ~~0.79~~ → 0.87

Change several columns at once: *SET price = 2.50, stock = 40*. The new value can use the old one (price * 1.10).

DELETE FROM: remove rows

```
DELETE FROM orders
WHERE order_id = 108;
```

In plain English: Anna cancelled her strawberries.

The whole row 108 is gone. DELETE always removes complete rows; to clear just one value, UPDATE it to NULL.

⚠ Forget WHERE and it hits EVERY row

UPDATE products SET price = 0; → every product is free.

DELETE FROM orders; → all orders gone. Usually there is no undo button.

Safe habit:

1. Write it as SELECT * … WHERE … and look at the rows.
2. Happy? Swap SELECT * for DELETE or UPDATE … SET.
3. Pros wrap risky changes in **BEGIN;** … then **COMMIT;** to keep or **ROLLBACK;** to undo.

ALTER TABLE: reshape a table

```
ALTER TABLE products
  ADD is_organic BOOLEAN DEFAULT FALSE;

ALTER TABLE products
  DROP COLUMN is_organic;
```

Add, remove or rename columns without rebuilding the table.

Three ways to "empty" things

command	what disappears
DELETE FROM t WHERE …	chosen rows
TRUNCATE TABLE t	all rows, table stays (fast)
DROP TABLE t	rows **and** the table itself

DROP DATABASE deletes everything. Handle with great care.

★ ***Takeaway:*** INSERT adds rows, UPDATE changes values, DELETE removes rows, ALTER reshapes, DROP destroys. Always check your WHERE with a SELECT first.

Part 5 · Building & changing

22 Handy extras: text, dates, views

Small tools you'll reach for all the time. Names vary a little between databases.

Text functions (applied to product_name)

function	does	example	gives
`UPPER(x)` / `LOWER(x)`	change case	UPPER('Bananas')	BANANAS
`LENGTH(x)`	count characters (SQL Server: LEN)	LENGTH('Bananas')	7
`CONCAT(a, b, ...)`	glue text together	CONCAT(product_name, ' (', category, ')')	Bananas (Fruit)
`SUBSTRING(x, start, n)`	cut out a piece	SUBSTRING('Bananas', 1, 3)	Ban
`REPLACE(x, old, new)`	swap text	REPLACE('Whole Milk', 'Whole ', '')	Milk
`TRIM(x)`	remove spaces at both ends	TRIM(' Apples ')	Apples

Numbers: `ROUND`(2.456, 2) → 2.46 · `CEIL`(2.1) → 3 · `FLOOR`(2.9) → 2 · `ABS`(−5) → 5

Dates

`CURRENT_DATE` → today, e.g. 2026-10-17

`EXTRACT(MONTH FROM order_date)` → 10 (MySQL & SQL Server also have *MONTH(order_date)*, *YEAR(...)*)

`DATEDIFF`('2026-10-07', '2026-10-01') → 6 days (MySQL; other databases phrase it differently)

Compare dates like numbers: later = bigger.

```
SELECT order_date, COUNT(*) AS orders
FROM orders
GROUP BY order_date
ORDER BY order_date;
```

10-01: **2** · 10-02: **1** · 10-03: **2** · 10-05: **1** · 10-06: **1** · 10-07: **1**
(no orders on 10-04, so no row for it)

VIEW: save a query under a name

```
CREATE VIEW low_stock AS
SELECT product_name, stock
FROM products
WHERE stock < 50;

SELECT * FROM low_stock;
```

A view behaves like a table but stores no data of its own. It re-runs the query each time, so it's always up to date. Great for reports people check daily.

INDEX: make searches faster

```
CREATE INDEX idx_product_name
  ON products (product_name);
```

Like the index at the back of a book: the database can jump straight to "Apples" instead of reading every row. Matters on big tables (millions of rows), not on our 12. Trade-off: inserts and updates get slightly slower. Primary keys get an index automatically.

Safety note: SQL injection

If an app glues a user's typed text straight into SQL, a sneaky user can type SQL code instead of a name and run it. Apps prevent this with **parameters** (placeholders like *WHERE first_name = ?*) so input is always treated as data, never as code. Worth knowing the name for interviews.

☆ **Takeaway:** functions reshape values (text, numbers, dates). A VIEW is a saved query; an INDEX is a shortcut for speed. Function names differ slightly by database: look them up when needed.

Part 6 · Review

23 The cheat sheet

Everything on one page. Print it and keep it next to your keyboard.

```
SELECT   columns, AGG(column) AS name   -- 5
FROM     table JOIN other ON a = b      -- 1
WHERE    row condition                  -- 2
GROUP BY columns                        -- 3
HAVING   group condition                -- 4
ORDER BY column DESC                    -- 6
LIMIT    n;                             -- 7
```

The master template. Write top to bottom. Grey numbers = the order SQL actually runs it. Every clause except SELECT and FROM is optional.

Reading data

SELECT a, b choose columns (* = all)	**AS nickname** rename a column or table	**DISTINCT** remove duplicate rows	**WHERE** keep rows that pass a test
= <> > < >= <= compare values	**AND / OR / NOT** both / either / flip. Use ( )	**IN ('a', 'b')** matches any in a list	**BETWEEN 1 AND 2** range, ends included
LIKE 'S%' % = any chars, _ = one char	**IS NULL** find missing values (never = NULL)	**ORDER BY x DESC** sort; ASC is default	**LIMIT n** first n rows (SQL Server: TOP n)

Summarising

COUNT(*) number of rows	**SUM / AVG** total / average	**MIN / MAX** smallest / biggest	**GROUP BY x** one result row per x
HAVING filter groups (after GROUP BY)	**CASE WHEN … END** if/then labels	**COALESCE(x, 0)** replace NULL with a default	**ROUND(x, 2)** round to 2 decimals

Combining

JOIN … ON a = b matches only (INNER)	**LEFT JOIN** all of 1st table + matches	**RIGHT / FULL JOIN** all of 2nd / all of both	**UNION (ALL)** stack results (keep repeats)
(SELECT …) subquery: runs first	**EXISTS (…)** is there at least one?	**WITH t AS (…)** name a subquery (CTE)	**CREATE VIEW** save a query as a "table"

Building & changing

CREATE TABLE columns + types + rules	**INSERT INTO … VALUES** add rows	**UPDATE … SET … WHERE** change values	**DELETE FROM … WHERE** remove rows

Question → SQL translator

"how many …"	→ COUNT(*)	"… per / for each …"	→ GROUP BY
"top 3 / cheapest 5"	→ ORDER BY + LIMIT	"only groups where …"	→ HAVING
"… and their names"	→ JOIN	"never / without any …"	→ LEFT JOIN + IS NULL
"above average …"	→ subquery	"starts with / contains"	→ LIKE

Part 6 · Review

24 Practice time

16 questions about FreshMart (page 2). Write the query, run it, then check page 25.

Level 1 · warm-up ★

1. List the name and price of every product. *p.5*
2. Which products are in the Snacks category? *p.7, quotes!*
3. Which products cost less than 1.50 €? *p.7*
4. Which customers live in Cologne? *different table*
5. Show all products, the most stock first. *p.11*

Level 2 · getting comfy ★★

6. Which products have a name starting with "C"? *LIKE, p.9*
7. Dairy or Bakery items with more than 50 in stock. *brackets! p.8*
8. Which product has no supplier recorded? *p.10*
9. What are the 3 cheapest products? *p.11*
10. How many products are actually in stock (stock above 0)? *p.12*

Level 3 · real analyst questions ★★★

11. Total stock per category, biggest first. *p.13*
12. Which categories have an average price above 1.85 €? *p.14, careful with Bakery*
13. Every order with the customer's first name and the product name. *3 tables, p.17*
14. How many products does each supplier provide, including suppliers with none? *LEFT JOIN + COUNT(column)*
15. Which customers have never placed an order? *p.16*
16. Bonus: what is FreshMart's total revenue from all 8 orders? *quantity × price*

25 Answers & results

Your query looks different but returns the same rows? Then it's right too.

1
```
SELECT product_name, price FROM products;
```
12 rows, one per product.

2
```
SELECT product_name FROM products
WHERE category = 'Snacks';
```
Dark Chocolate, Potato Chips

3
```
SELECT product_name, price FROM products
WHERE price < 1.50;
```
6 rows: Bananas, Whole Milk, Croissant, Sparkling Water, Potato Chips, Greek Yogurt

4
```
SELECT first_name FROM customers
WHERE city = 'Cologne';
```
Ben, Emma

5
```
SELECT product_name, stock FROM products
ORDER BY stock DESC;
```
Sparkling Water 200, Bananas 150, Apples 120 … last: Orange Juice 0

6
```
SELECT product_name FROM products
WHERE product_name LIKE 'C%';
```
Cheddar Cheese, Croissant

7
```
SELECT product_name, stock FROM products
WHERE (category = 'Dairy' OR category =
'Bakery')
  AND stock > 50;
```
Whole Milk 80, Croissant 60. Greek Yogurt has exactly 50: not "more than".

8
```
SELECT product_name FROM products
WHERE supplier_id IS NULL;
```
Dark Chocolate

9
```
SELECT product_name, price FROM products
ORDER BY price LIMIT 3;
```
Sparkling Water 0.60, Greek Yogurt 0.79, Croissant 0.90

10
```
SELECT COUNT(*) FROM products
WHERE stock > 0;
```
11 (only Orange Juice is sold out)

11
```
SELECT category, SUM(stock) AS total
FROM products GROUP BY category
ORDER BY total DESC;
```
Fruit 300, Drinks 200, Dairy 170, Snacks 165, Bakery 85

12
```
SELECT category, AVG(price) FROM products
GROUP BY category
HAVING AVG(price) > 1.85;
```
Fruit 2.53, Drinks 1.90. Bakery is exactly 1.85, so not "above".

13
```
SELECT o.order_id, c.first_name, p.product_name
FROM orders o
JOIN customers c ON o.customer_id =
c.customer_id
JOIN products p ON o.product_id = p.product_id;
```
8 rows: 101 Anna Bananas, 102 Anna Whole Milk, 103 Ben Sourdough Bread … (p.17)

14
```
SELECT s.supplier_name, COUNT(p.product_id)
AS n
FROM suppliers s
LEFT JOIN products p ON s.supplier_id =
p.supplier_id
GROUP BY s.supplier_name;
```
Green Farm 3, Alpine Dairy 3, Baker's Best 2, Splash Drinks 2, Crunch Co 1, Nordic Fish 0. COUNT(*) would wrongly give Nordic Fish 1 (it counts the NULL row).

15
```
SELECT c.first_name FROM customers c
LEFT JOIN orders o ON c.customer_id =
o.customer_id
WHERE o.order_id IS NULL;
```
Emma

16
```
SELECT SUM(o.quantity * p.price) AS revenue
FROM orders o
JOIN products p ON o.product_id = p.product_id;
```
33.47 € (13.39 + 10.18 + 6.40 + 3.50, see p.17)

You made it through all 25 pages! Next step: invent your own FreshMart questions ("which city spends most?") and answer them. That's exactly what a data analyst does all day.